

Phần 1 - Giới thiệu Software Architecture

Mục lục

Software Architecture Foundation	3
Vì sao có khoá này?	3
Mạch logic của khoá	3
Đối tượng người đọc	4
Nếu bạn đến từ Data Science	4
Cấu trúc tài liệu	4
Cách đọc tài liệu	5
Người đọc lần đầu	5
Data Scientist muốn productionize ML	5
Người đã có kinh nghiệm và muốn tra cứu nhanh	6
Triết lý biên soạn	6
Quy ước hiển thị	6
Bắt đầu từ đâu?	6
1.1 Tổng quan Phần 1: Giới thiệu Software Architecture	7
Nếu bạn đến từ Data Science	7
Vì sao Phần 1 cần đứng trước	7
Bốn bài trong phần	7
Bài 1.2: Software Architecture là gì?	7
Bài 1.3: Mục tiêu và kết quả học tập	7
Bài 1.4: Lộ trình đọc tài liệu	7
Cách đọc Phần 1	8
Kết nối với các phần sau	8
Sai lầm cần tránh ngay từ đầu	8
1.2 Software Architecture là gì?	9
Nếu bạn đến từ Data Science	9
Một câu hỏi đơn giản, ba câu trả lời khác nhau	9
Góc nhìn 1: Định nghĩa hình thức (Bass-Clements-Kazman)	9
Góc nhìn 2: Định nghĩa thực dụng (Martin Fowler)	9
Góc nhìn 3: Định nghĩa operational (cái gì architect thực sự làm)	10
Tổng hợp: Định nghĩa “operational” của tài liệu này	10
Ba ví dụ minh hoạ	10
Ví dụ 1: Chọn dùng PostgreSQL hay MongoDB cho hệ thống mại điện tử	10
Ví dụ 2: Đặt tên biến userCount hay numUsers trong một function	11
Ví dụ 3: Dùng JWT hay session cookie cho authentication trong một SaaS B2B	11
“Architecture is design”, nhưng không phải mọi design đều architectural	11

Vì sao kiến trúc lại đắt khi sửa?	11
1. Cascade effect	12
2. Conway's law	12
3. Sunk cost của technology lock-in	12
Một “architect” thực sự làm gì?	12
Tóm tắt	12
1.3 Mục tiêu và kết quả học tập	13
Nếu bạn đến từ Data Science	13
Vì sao cần nói rõ mục tiêu?	13
Mục tiêu khoá (Aims)	13
Mục tiêu 1: Nhận biết và lập luận về architectural decisions	13
Mục tiêu 2: Áp dụng SOLID và các nguyên lý thiết kế cấp module	13
Mục tiêu 3: Biết 9 architecture styles phổ biến	13
Mục tiêu 4: Đọc và viết documentation kiến trúc chuẩn	14
Mục tiêu 5: Áp dụng vào case study thực	14
Mục tiêu 6: Áp dụng nền tảng vào advanced domains	14
Kết quả học tập (Learning Outcomes)	14
Không nằm trong scope	15
1. Domain-Driven Design (DDD)	15
2. CQRS, Event Sourcing, Saga, Outbox pattern	15
3. Hexagonal / Clean / Onion Architecture	15
4. Cloud-native specifics	15
5. Performance engineering chi tiết	16
6. Security architecture chi tiết	16
Lộ trình học tiếp sau khoá	16
Nếu bạn đi hướng Architect chuyên nghiệp	16
Nếu bạn đi hướng Tech Lead	16
Nếu bạn đi hướng Distributed Systems	16
Nếu bạn đi hướng Cloud Architect	16
Tóm tắt	16
1.4 Lộ trình đọc tài liệu	17
Trước khi chọn lộ trình	17
Sơ đồ phụ thuộc giữa các phần	17
Lộ trình A: Chuyên sâu đầy đủ	17
Lộ trình B: Data Scientist to Software Architecture	18
Vì sao path này khác?	18
Lộ trình C: Refresh nhanh	19
Lộ trình D: Tra cứu on-demand	19
Shortcut paths theo focus area	19
Shortcut 1: Tôi muốn hiểu microservices	19
Shortcut 2: Tôi muốn document architecture cho team	20
Shortcut 3: Tôi muốn refactor code DS/ML cho dễ maintain	20
Shortcut 4: Tôi muốn thiết kế production ML system	20
Lưu ý khi học	20
Đừng chỉ đọc, hãy vẽ	20
Đừng bị quyến rũ bởi trends	21
Quay lại Phần 4 nhiều lần	21

Software Architecture Foundation

Chào mừng bạn đến với **Software Architecture Foundation**, một bộ tài liệu mở Tiếng Việt về Kiến trúc Phần mềm. Tài liệu này không cố gắng thay thế các sách kinh điển như *Clean Architecture* hay *Software Architecture in Practice*. Nó đóng vai trò như một người hướng dẫn kiên nhẫn: giải thích trực giác trước, formalize sau, rồi gắn từng khái niệm với ví dụ đủ thực tế để bạn áp dụng được vào dự án.

Nếu bạn đến từ backend hoặc software engineering, nhiều ví dụ sẽ rất quen: API, database, microservices, deployment, monitoring. Nếu bạn đến từ **Data Science**, tài liệu này cũng dành cho bạn. Khi một model rời notebook để đi vào production, bạn không còn chỉ làm modelling nữa. Bạn đang xây một hệ thống phần mềm có data pipeline, feature store, model registry, serving endpoint, monitoring, alerting, rollback và audit trail. Đó chính là lãnh thổ của Software Architecture.

Vì sao có khoá này?

Software Architecture là một trong những lĩnh vực mà người mới rất dễ bị ngợp. Mở một quyển sách bất kỳ, bạn sẽ gặp hàng chục thuật ngữ: SOLID, ATAM, 4+1 views, layered, hexagonal, microservices, event-driven, CQRS, saga, BFF, service mesh. Mỗi thuật ngữ lại kéo theo một họ kỹ thuật con. Người học thường không biết đâu là cốt lõi, đâu là biến thể, đâu là trend, và đâu là thứ thật sự giúp mình ra quyết định tốt hơn.

Với Data Scientist, cảm giác này còn rõ hơn. Bạn có thể đã rất giỏi pandas, scikit-learn, PyTorch, SQL, Airflow hoặc dbt, nhưng khi nghe “bounded context”, “deployment view”, “quality attribute”, “distributed transaction”, bạn vẫn thấy như đang học một ngôn ngữ khác. Vấn đề không nằm ở năng lực của bạn. Vấn đề là nhiều tài liệu Software Architecture được viết ngầm định rằng người đọc đã sống nhiều năm trong backend engineering.

Khoá này cố gắng xây cây cầu giữa hai thế giới đó. Ta sẽ vẫn học Software Architecture nghiêm túc, nhưng khi cần, ta sẽ dùng các ví dụ quen thuộc với Data Scientist:

- Notebook training script biến thành pipeline production.
- Batch inference so với online inference.
- Feature store và data ownership.
- Model serving latency, throughput và cost GPU.
- Data drift, model drift, data lineage và reproducibility.
- ML monitoring, rollback và auditability.

Mạch logic của khoá

Toàn bộ nội dung được sắp xếp theo một mạch rất cụ thể:

1. **Bắt đầu từ code tốt:** Phần 2 dạy cohesion, coupling và SOLID. Với Data Scientist, đây là bước chuyển từ notebook hoặc script dài sang code có module rõ ràng.
2. **Lùi lại nhìn bức tranh lớn:** Phần 3 dạy phân biệt architecture decision với design decision, và học cách phân tích trade-off.
3. **Hiểu cái cần tối ưu:** Phần 4 dạy Quality Attributes. Đây là phần cực quan trọng cho production ML vì accuracy không phải metric duy nhất.
4. **Học các style tổ chức hệ thống:** Phần 5 và 6 đi từ monolith, layered, pipeline tới service-based, microservices và event-driven.
5. **Biết cách truyền đạt:** Phần 7 dạy cách vẽ view và viết ADR. Kiến trúc không được truyền đạt rõ thì rất khó vận hành lâu dài.
6. **Áp vào case study:** Phần 8 cho thấy cách lập luận end-to-end trong các hệ thật.

Nói ngắn gọn: khoá không dạy bạn thuộc lòng pattern. Khoá dạy bạn nhìn một bài toán, hỏi đúng câu hỏi, chọn đúng trade-off, rồi giải thích quyết định đó cho người khác.

Đối tượng người đọc

Khoá này phù hợp với bốn nhóm chính:

- **Data Scientist / ML Engineer / Data Engineer** muốn hiểu cách đưa model, feature pipeline và data product vào production một cách bền vững.
- **Lập trình viên 2-5 năm kinh nghiệm** đang chuẩn bị bước lên vai trò Senior Engineer, Tech Lead hoặc Architect.
- **Học viên cao học ngành Khoa học Máy tính / Kỹ thuật Phần mềm** cần một tài liệu Tiếng Việt có cấu trúc rõ ràng.
- **Người tự học** muốn lấp khoảng trống giữa “biết viết code” và “biết thiết kế hệ thống”.

Bạn không cần phải là backend expert trước khi đọc. Tuy nhiên, bạn nên có một số nền tảng tối thiểu:

- Đã viết code đủ nhiều để hiểu cảm giác code khó sửa, khó test, khó debug.
- Biết cơ bản về HTTP/API, database SQL/NoSQL và version control.
- Nếu chưa quen OOP, class, interface, inheritance, các bài SOLID có thể cần đọc chậm hơn. Khi đó, hãy dùng các ví dụ Python/ML trong tài liệu như điểm tựa.

Nếu bạn đến từ Data Science

Hãy đọc khoá này với một câu hỏi xuyên suốt: **điều gì làm một model sống được trong production?**

Trong notebook, bạn thường tối ưu metric offline. Trong production, bạn phải tối ưu cả hệ thống:

Trong notebook	Trong production
Accuracy, F1, AUC	Latency, throughput, reliability, rollback
CSV hoặc dataframe local	Data contract, schema, lineage, freshness
Một người chạy experiment	Nhiều team cùng vận hành pipeline
pickle hoặc artifact local	Model registry, versioning, deployment strategy
Print/log thủ công	Metrics, traces, dashboards, alerts
Chạy lại khi cần	Scheduled job, retry, idempotency, monitoring

Software Architecture không thay thế modelling. Nó giúp modelling trở thành một phần của sản phẩm thật.

Nếu bạn muốn lộ trình riêng, hãy mở [Lộ trình cho Data Scientist](#). Nếu muốn xem review nội dung theo góc nhìn DS, xem [Review nội dung cho Data Scientist](#).

Cấu trúc tài liệu

Khoá gồm **chín phần bài giảng** và **các tài nguyên tra cứu**.

Phần	Tên	Số bài	Trọng tâm
1	Giới thiệu Software Architecture	4	SA là gì, vì sao học, lộ trình
2	Design Principles (SOLID)	7	Cohesion, coupling, 5 nguyên lý SOLID

Phần	Tên	Số bài	Trọng tâm
3	Architectural Thinking	4	Architecture vs Design, trade-off, modularity
4	Quality Attributes	4	NFR, architecture characteristics, ATAM, component thinking
5	Fundamental Styles	5	Monolith, layered, pipeline, microkernel
6	Distributed Styles	5	Service-based, microservices, event-driven, space-based
7	Documenting Architecture	4	Module/C&C/Allocation views, ADR
8	Case Studies	5	UAMS, Smart City, Production ML Feature Store, bài tập tổng hợp
9	Seminar - Advanced Topics	5	IoT, Web3, MLOps, Digital Twin

Mỗi phần bắt đầu bằng một bài overview. Nếu bạn đọc tuần tự, overview giúp bạn biết phần này giải quyết vấn đề gì. Nếu bạn đọc tra cứu, overview giúp bạn nhanh chóng định vị bài cần đọc.

Cách đọc tài liệu

Người đọc lần đầu

Đọc theo thứ tự tự nhiên: Phần 1 □ 2 □ 3 □ 4 □ 5 □ 6 □ 7 □ 8 □ 9. Đây là đường chắc nhất nếu bạn muốn xây nền tảng đầy đủ. Sau Phần 4, bạn đã hiểu vì sao kiến trúc bị chi phối bởi Quality Attributes. Sau Phần 6, bạn có đủ vocabulary để phân tích đa số system design ở mức trung bình. Sau Phần 7 và 8, bạn biết cách trình bày lựa chọn kiến trúc cho team. Phần 9 giúp bạn áp dụng nền tảng đó vào các domain hiện đại khó hơn như IoT, Web3, MLOps và Digital Twin.

Data Scientist muốn productionize ML

Đừng bắt đầu bằng microservices. Hãy đi theo path này:

1. Bài 1.2 để hiểu quyết định nào là architectural.
2. Bài 3.3 để học trade-off.
3. Phần 4 để chuyển từ model metric sang system metric.
4. Bài 5.4 để hiểu pipeline architecture.
5. Bài 6.4 để hiểu event-driven và streaming.
6. Phần 7 để biết cách document ML/data systems.
7. Bài 8.3 để xem case real-time data/ML system.
8. Bài 9.4 để xem MLOps như một production platform end-to-end.

Path chi tiết nằm ở [Lộ trình cho Data Scientist](#).

Người đã có kinh nghiệm và muốn tra cứu nhanh

Vào thẳng [Glossary](#), [Course Summary](#), hoặc [Cross-reference](#). Mỗi bài cố gắng tự đứng được, nên bạn có thể mở đúng chủ đề đang cần.

Triết lý biên soạn

Bốn nguyên tắc xuyên suốt:

1. **Trực giác trước hình thức.** Ta bắt đầu bằng câu hỏi đời thực, rồi mới đưa định nghĩa.
2. **Ví dụ cụ thể hơn khẩu hiệu.** Một thuật ngữ chỉ thật sự có ý nghĩa khi bạn thấy nó giải quyết vấn đề nào.
3. **Trade-off luôn rõ ràng.** Không có best practice tuyệt đối. Mỗi quyết định đều mua một thứ bằng cách hy sinh thứ khác.
4. **Tài liệu hoá là first-class.** Kiến trúc trong đầu một người là rủi ro. Kiến trúc cần được vẽ, viết, review và cập nhật.

Quy ước hiển thị

- **Tóm tắt một dòng:** ý chính của bài, đọc khi bạn chỉ có 30 giây.
- **Nếu bạn đến từ Data Science:** cầu nối từ DS/ML sang Software Architecture.
- **Sai lầm thường gặp:** các cách hiểu sai dễ gặp.
- **Code block:** ví dụ code hoặc pseudo-code có chủ đích.
- **Mermaid diagram:** sơ đồ topology/flow render trực tiếp trên web.
- **Khi nào dùng / khi nào không:** bảng giúp ra quyết định nhanh.

Bắt đầu từ đâu?

Nếu bạn đọc lần đầu, bắt đầu với [Phần 1: Tổng quan](#). Nếu bạn đến từ Data Science và muốn đi đường ngắn hơn, mở ngay [Lộ trình cho Data Scientist](#). Nếu bạn đã biết Software Architecture và chỉ cần tra cứu, bắt đầu từ [Course Summary](#) hoặc [Glossary](#).

Chúc bạn học hiệu quả và biến được mô hình, pipeline, service của mình thành hệ thống production đáng tin cậy.

1.1 Tổng quan Phần 1: Giới thiệu Software Architecture

Tóm tắt một dòng: Software Architecture là tập hợp các quyết định khó-thay-đổi-nhất của một hệ phần mềm, và phần này dạy bạn cách nhận ra những quyết định đó, đặt tên cho chúng, và bắt đầu cân nhắc chúng một cách có hệ thống.

Nếu bạn đến từ Data Science

Phần 1 nên được đọc như phần định nghĩa lại phạm vi công việc khi model đi vào production. Trong notebook, bạn thường kiểm soát toàn bộ môi trường. Trong production, model phụ thuộc vào API, data pipeline, feature store, database, scheduler, monitoring, alerting và quy trình rollback. Phần này giúp bạn gọi đúng tên các quyết định quan trọng đó.

Vì sao Phần 1 cần đứng trước

Có một sai lầm rất phổ biến khi học Software Architecture: nhảy ngay vào học microservices, event-driven, hay DDD vì nghe có vẻ “kiến trúc”. Hậu quả là người học thuộc nhiều pattern nhưng không biết khi nào dùng, không biết vì sao pattern này lại sinh ra, và quan trọng nhất, không biết phân biệt vấn đề kiến trúc với vấn đề thiết kế thuần túy. Phần 1 sửa sai lầm này bằng cách bắt đầu từ câu hỏi cơ bản nhất: “Software Architecture rốt cuộc là gì?”.

Sau khi đọc xong phần này, bạn sẽ:

1. Phân biệt được Architecture với Design ở mức câu hỏi (Architecture trả lời “shape of system”, Design trả lời “shape of code inside a module”).
2. Hiểu vì sao những quyết định kiến trúc lại đắt khi sửa, và do đó vì sao chúng đáng để bỏ thời gian cân nhắc kỹ.
3. Có một lộ trình rõ ràng cho 7 phần còn lại: phần nào trả lời câu hỏi gì.
4. Biết cách dùng tài liệu này hiệu quả nhất theo nhu cầu (đọc lần đầu, ôn thi, hay tra cứu).

Bốn bài trong phần

Phần 1 ngắn nhất khoá, chỉ có 4 bài, vì mục tiêu là “khởi động” chứ không phải “đi sâu”. Đào sâu sẽ bắt đầu từ Phần 2.

Bài 1.2: Software Architecture là gì?

Bài này định nghĩa Software Architecture qua ba góc nhìn bổ sung cho nhau: định nghĩa hình thức (theo Bass-Clements-Kazman), định nghĩa thực dụng (theo Martin Fowler), và định nghĩa “operational” (cái gì bạn phải làm khi bạn là một architect). Đặc biệt nhấn mạnh điểm: architecture không phải là “cái diagram đẹp”, mà là *tập hợp các quyết định khó sửa*.

Bài 1.3: Mục tiêu và kết quả học tập

Bài này nói rõ khoá học định cho bạn cái gì khi kết thúc: bạn sẽ biết tên gọi và bản chất 9 architecture styles phổ biến nhất, có thể đọc một bộ documentation kiến trúc bất kỳ và nhận ra đó là kiểu view nào, và đặc biệt là biết cách lập luận trade-off khi chọn style cho dự án mới. Bài cũng giải thích vì sao khoá không dạy *tất cả* (CQRS, Event Sourcing, Saga, hexagonal... đều bị bỏ ra) và bạn nên học gì tiếp theo.

Bài 1.4: Lộ trình đọc tài liệu

Bài này vẽ một sơ đồ phụ thuộc giữa các phần, đề xuất 3 lộ trình khác nhau (chuyên sâu, ôn thi, tra cứu), và gợi ý thời gian ước lượng cho mỗi lộ trình. Đặc biệt có “shortcut paths” cho ai chỉ quan tâm tới một

focus area cụ thể (vd: chỉ muốn học microservices, hoặc chỉ muốn ôn cho phỏng vấn).

Cách đọc Phần 1

Phần 1 dài tổng cộng chỉ khoảng 6000-8000 chữ. Đọc nghiêm túc mất khoảng 1-1.5 giờ. Đây là phần bạn nên đọc *tuần tự* và *không skip*, vì nó set lên framework tư duy cho toàn khoá. Đặc biệt Bài 1.2 là bài quan trọng nhất, nếu bạn chỉ đọc một bài trong cả khoá, hãy đọc bài đó.

Sau khi đọc xong Phần 1, bạn nên thấy một sự “click”: các thuật ngữ và khái niệm rời rạc bạn từng nghe sẽ bắt đầu có chỗ đứng trong một bản đồ chung. Nếu chưa thấy click, đừng vội đi tiếp, đọc lại Bài 1.2 và Bài 1.4 thêm một lần.

Kết nối với các phần sau

Phần 1 không có nội dung kỹ thuật, nó chỉ làm nhiệm vụ “đặt khung”. Toàn bộ kiến thức kỹ thuật bắt đầu từ Phần 2:

- **Phần 2 (SOLID)** trả lời câu hỏi: “Kiến trúc tốt bắt đầu từ đâu?”, câu trả lời là “từ code tốt”.
- **Phần 3 (Architectural Thinking)** mở rộng từ code lên hệ thống: “Khi nào một quyết định trở thành kiến trúc?”.
- **Phần 4 (Quality Attributes)** trả lời: “Tối ưu cái gì?”, vì bạn không thể tối ưu mọi thứ cùng lúc.
- **Phần 5-6 (Architecture Styles)** dạy các “công cụ” chuẩn: 9 cách phổ biến để tổ chức một hệ thống.
- **Phần 7 (Documenting)** dạy cách truyền đạt: kiến trúc không nói ra được là kiến trúc chết.
- **Phần 8 (Case Studies)** áp dụng toàn bộ vào case thật.

Sai lầm cần tránh ngay từ đầu

Trước khi sang Bài 1.2, hãy ghi nhớ ba sai lầm phổ biến nhất khi tiếp cận Software Architecture:

1. **Coi architecture là “high-level design”**. Sai. Architecture không phải là design ở zoom level lớn hơn. Architecture là *một loại* quyết định khác, đặc trưng bởi tính khó-thay-đổi và ảnh hưởng tới quality attributes của toàn hệ. Bài 1.2 và Phần 3 sẽ làm rõ điểm này.
2. **Coi architecture là “việc của architect”**. Sai. Trong một team trưởng thành, mọi engineer đều cần hiểu kiến trúc của hệ mình làm, chỉ là họ không tự quyết những thứ ở mức kiến trúc thôi. Bài 1.3 sẽ nói rõ.
3. **Coi architecture là cuộc đua đến microservices**. Sai cực kỳ phổ biến trong 5 năm gần đây. Microservices là *một style*, có chỗ dùng đúng và rất nhiều chỗ dùng sai. Phần lớn dự án bắt đầu nên là monolith (Phần 5 sẽ giải thích). Đừng bị quyến rũ bởi pattern thời thượng, phải hiểu trade-off trước.

Nếu bạn đã thấy mình đang phạm một trong ba sai lầm trên, đừng lo. Hết Phần 1 bạn sẽ gỡ được tất cả. Cùng bắt đầu với [Bài 1.2: Software Architecture là gì?](#).

1.2 Software Architecture là gì?

Tóm tắt một dòng: Software Architecture là tập hợp các quyết định khó-thay-đổi-nhất của một hệ phần mềm, đặc trưng bởi (a) tính bao trùm toàn hệ thống, (b) ảnh hưởng quyết định tới quality attributes, và (c) chi phí sửa cao tới mức phải cân nhắc rất kỹ ngay từ đầu.

Nếu bạn đến từ Data Science

Một cách rất thực tế để hiểu Software Architecture là hỏi: quyết định này có làm thay đổi cách model sống trong production không? Nếu bạn đổi `max_depth` của XGBoost từ 6 sang 8, đó thường là design hoặc experiment detail. Nhưng nếu bạn quyết định batch inference mỗi đêm thay vì online inference theo request, đó là architectural decision: nó ảnh hưởng latency, cost, data freshness, monitoring, deployment và cách team vận hành.

Tương tự, chọn lưu feature trong warehouse hay dùng feature store không chỉ là chọn tool. Nó quyết định data ownership, consistency giữa training và serving, khả năng audit prediction, và cách debug khi feature bị drift. Những quyết định như vậy thường khó đổi sau khi hệ đã chạy, nên chúng thuộc về Software Architecture.

Một câu hỏi đơn giản, ba câu trả lời khác nhau

Nếu bạn hỏi ba kỹ sư phần mềm “Software Architecture là gì?”, rất có thể bạn nhận được ba câu trả lời khác nhau và cả ba đều đúng một phần. Đó không phải lỗi của họ, đó là vì khái niệm này có nhiều lớp ý nghĩa, và mỗi cộng đồng (academia, industry, một dự án cụ thể) nhấn mạnh một lớp khác nhau. Bài này sẽ đi qua ba góc nhìn phổ biến nhất, sau đó tổng hợp lại thành một định nghĩa hành nghề được.

Góc nhìn 1: Định nghĩa hình thức (Bass-Clements-Kazman)

Trong sách kinh điển *Software Architecture in Practice* (xuất bản lần đầu 1998, đến nay đã ấn bản thứ tư), nhóm tác giả Bass-Clements-Kazman đưa ra định nghĩa sau:

“The software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.”

Diễn ra Tiếng Việt và bỏ formal-talk:

- Architecture là **một tập hợp các structure** (cấu trúc), không phải một structure duy nhất. Một hệ thống có nhiều cấu trúc song song: cấu trúc module, cấu trúc runtime, cấu trúc deployment...
- Mỗi structure gồm **elements** (thành phần) và **relations** (quan hệ giữa chúng).
- Cả elements và relations đều có **properties** (tính chất) đáng quan tâm.

Định nghĩa này tốt vì nó *bao quát*: bất cứ cái gì giúp bạn lý giải về hệ thống đều có thể coi là một phần của architecture. Nhưng nó hơi mơ hồ cho người mới: “structure nào quan trọng” thì câu này không trả lời.

Góc nhìn 2: Định nghĩa thực dụng (Martin Fowler)

Trong bài *Who Needs an Architect?* (2003), Martin Fowler, một trong những voice ảnh hưởng nhất ngành Software, phỏng vấn nhiều architect đầu ngành và rút ra:

“Architecture is about the important stuff. Whatever that is.”

Câu này nghe đùa nhưng cực kỳ sâu. Ý của Fowler:

- Architecture không phải là một loại đối tượng cụ thể (không phải “diagram”, không phải “module list”, không phải “API spec”).
- Architecture là **whatever decisions are hard to change later**. Cái gì khó sửa, cái đó là kiến trúc. Cái gì dễ sửa, cái đó là thiết kế chi tiết hoặc implementation.
- “Important” ở đây không phải “to bigger” hay “more abstract”, mà là **important to the long-term success** của hệ thống.

Định nghĩa này thực dụng hơn nhiều, và là cách phần lớn architect industry hiểu khái niệm này. Nhưng nó vẫn cần thêm tiêu chí cụ thể để áp dụng.

Góc nhìn 3: Định nghĩa operational (cái gì architect thực sự làm)

Nếu nhìn vào công việc hàng ngày của một software architect ở công ty trưởng thành, bạn sẽ thấy họ thực sự dành thời gian cho:

1. **Lựa chọn architecture style cho hệ mới**: monolith hay microservices? Layered hay event-driven? Style nào phù hợp với quality attributes ưu tiên?
2. **Định nghĩa boundary giữa các component**: cái gì là module riêng, cái gì là sub-module, cái gì là phần của một module lớn hơn?
3. **Quyết định technology stack chiến lược**: Java vs Go vs Node? PostgreSQL vs MongoDB? Kafka hay RabbitMQ?
4. **Trade-off khi xung đột quality attributes**: chọn performance hay maintainability? Chọn consistency hay availability?
5. **Documenting và truyền đạt**: viết Architecture Decision Records (ADR), vẽ diagrams chuẩn, review design của team.
6. **Cross-team coordination**: đảm bảo các team build các phần khác nhau của hệ vẫn ghép được với nhau.

Tất cả 6 việc trên đều có một đặc điểm chung: **quyết định một lần ảnh hưởng dài hạn, sửa lại thì rất đắt**. Đó chính là essence của architecture.

Tổng hợp: Định nghĩa “operational” của tài liệu này

Trong toàn bộ khoá học, chúng ta sẽ dùng định nghĩa sau:

Software Architecture là tập hợp các quyết định thiết kế của một hệ phần mềm thoả mãn cả ba tiêu chí: 1. **Bao trùm toàn hệ thống** (system-wide), không thuộc về một module riêng lẻ. 2. **Ảnh hưởng quyết định tới quality attributes** (performance, scalability, security, maintainability...) thay vì chỉ tới functional behavior. 3. **Chi phí thay đổi cao**, sửa sau khi đã code/deploy rất tốn kém.

Bất cứ quyết định nào đạt cả ba tiêu chí trên là kiến trúc. Cái gì không đạt là design thông thường hoặc implementation detail.

Ba ví dụ minh họa

Để định nghĩa trên trở nên cụ thể, hãy xét ba quyết định và xác định cái nào là architectural, cái nào không.

Ví dụ 1: Chọn dùng PostgreSQL hay MongoDB cho hệ thương mại điện tử

- Bao trùm toàn hệ? ☐ (mọi service đều dùng tới database này).

- Ảnh hưởng quality attribute? ☐ (consistency, query expressiveness, scalability đều khác nhau giữa hai lựa chọn).
- Chi phí thay đổi cao? ☐ (migrate database sau 2 năm production là một dự án 6-12 tháng).

☐ **Architectural decision.** Phải được cân nhắc kỹ ngay đầu dự án.

Ví dụ 2: Đặt tên biến `userCount` hay `numUsers` trong một function

- Bao trùm toàn hệ? ☐ (chỉ trong scope một function).
- Ảnh hưởng quality attribute? ☐ (cùng lắm là code style).
- Chi phí thay đổi? ☐ (IDE rename, một phím tắt).

☐ **Implementation detail.** Đừng tranh cãi tới 30 phút trong code review.

Ví dụ 3: Dùng JWT hay session cookie cho authentication trong một SaaS B2B

- Bao trùm toàn hệ? ☐ (mọi protected endpoint đều phải verify).
- Ảnh hưởng quality attribute? ☐ (scalability, security, sessions revocation đều khác).
- Chi phí thay đổi? ☐ (medium, có thể migrate dần qua dual support nhưng vẫn cần effort).

☐ **Architectural** (tier 2). Cần thảo luận team trước khi chốt.

Bài tập nhỏ cho bạn: đặt 5 quyết định gần đây trong dự án bạn đang làm vào framework 3 tiêu chí này. Bạn sẽ thấy phần lớn quyết định *không* là architectural (mặc dù cãi nhau cũng nhiều). Chỉ một số nhỏ thực sự deserve mức attention architectural.

“Architecture is design”, nhưng không phải mọi design đều architectural

Có một câu nổi tiếng của Grady Booch:

“All architecture is design, but not all design is architecture.”

Hiểu về đầu: architecture cũng là một dạng design, không có gì huyền bí. Nó vẫn là quyết định “structure this way vs that way”. Hiểu về sau: nhưng chỉ một tập con của design quyết định mới đáng được gọi là architecture, tập con đạt ba tiêu chí ở trên.

Hệ quả: ranh giới giữa architecture và design *không cố định*, nó phụ thuộc context. Cùng một quyết định có thể là architectural trong project A nhưng là implementation detail trong project B. Ví dụ:

- Trong một startup 5 người làm MVP: cấu trúc class hierarchy của một module là implementation detail (vì cả team thuộc, sửa nhanh).
- Trong một enterprise 200 engineers: cùng quyết định đó có thể là architectural (vì 50 người sẽ depend, sửa cần coordinate).

Điều này dẫn tới một insight quan trọng: **vai trò architect không phải để “ra quyết định đúng” mà là “nhận ra quyết định nào quan trọng đến mức cần được xem xét nghiêm túc”.**

Vì sao kiến trúc lại đắt khi sửa?

Có ba lý do chính khiến architectural decisions đắt khi sửa, và hiểu rõ ba lý do này giúp bạn cẩn thận hơn ngay từ đầu:

1. Cascade effect

Một quyết định architectural thường được nhiều phần khác depend vào. Đổi database từ MongoDB sang PostgreSQL không chỉ là rewrite data access layer, bạn còn phải đổi schema design, query patterns, ORM, deployment, monitoring, backup strategy, on-call runbook... Mỗi cái lại depend vào hàng tá thứ khác. Cascade này có thể mất hàng tháng untangle.

2. Conway's law

Architecture của hệ phản chiếu organization structure của team build nó. Một khi team đã được tổ chức xung quanh một architecture nhất định (vd: 5 team mỗi team own một microservice), đổi architecture thường đồng nghĩa đổi org chart, việc cực kỳ tốn về mặt chính trị và con người. Conway's law sẽ được nhắc lại nhiều lần trong khoá.

3. Sunk cost của technology lock-in

Mỗi technology choice tích lũy “muscle memory” của team: tools, libraries, internal frameworks, training, hiring criteria... Đổi tech stack đồng nghĩa team phải học lại, viết lại tooling, có khi tuyển người mới. Sunk cost này tăng tuyến tính với tuổi project.

Một “architect” thực sự làm gì?

Tài liệu này tránh dùng từ “architect” như một chức danh, vì:

- Ở một số công ty, architect là chức danh chính thức (vd: Solution Architect, Enterprise Architect).
- Ở công ty khác, không có chức danh này, quyết định kiến trúc được chia cho Tech Lead, Staff Engineer, hoặc CTO.
- Ở startup, một developer 5 năm kinh nghiệm có thể đã đang làm việc của một architect mà không gọi tên thế.

Vai trò “architect”, bất kể chức danh, bao gồm:

1. **Phân biệt architectural decision với design decision:** dành thời gian cho cái thứ nhất, để cái thứ hai cho engineer thực thi tự quyết.
2. **Cân nhắc trade-off đa chiều:** không bao giờ có “best architecture”, chỉ có “best for current context”.
3. **Truyền đạt và document:** để team hiểu *vì sao* quyết định kia, không chỉ *cái gì* được quyết.
4. **Sẵn sàng review và adjust:** kiến trúc evolve, không có “set and forget”.
5. **Code đủ để hiểu pain:** architect xa rời code lâu sẽ ra quyết định lý thuyết, không thực dụng. Phần 3 sẽ nói rõ.

Tóm tắt

- Software Architecture là tập hợp **quyết định khó-sửa-nhất** ảnh hưởng toàn hệ và quality attributes.
- Có ba định nghĩa phổ biến (Bass-Clements-Kazman, Fowler, operational), và cả ba bổ sung nhau.
- Phân biệt architectural decisions với implementation details bằng **3 tiêu chí**: bao trùm, ảnh hưởng QA, đắt khi sửa.
- Architecture đắt khi sửa do **cascade effect**, **Conway's law**, và **technology lock-in**.
- “Architect” là một *vai trò*, không phải bắt buộc một *chức danh*.

Bài tiếp theo: [Mục tiêu và kết quả học tập](#), khoá này định cho bạn cái gì khi kết thúc.

1.3 Mục tiêu và kết quả học tập

Tóm tắt một dòng: Khoá này muốn bạn ra trường biết tên 9 architecture styles phổ biến, đọc được documentation kiến trúc bất kỳ, và lập luận được trade-off khi chọn architecture cho dự án mới, chứ không cố làm bạn thành expert ở mọi pattern.

Nếu bạn đến từ Data Science

Sau khoá này, mục tiêu không phải biến bạn thành backend architect ngay lập tức. Mục tiêu thực tế hơn là giúp bạn nói chuyện được với platform/backend team bằng cùng một vocabulary. Khi họ hỏi batch hay online inference, REST hay Kafka, model registry tự build hay dùng managed service, bạn hiểu câu hỏi đó liên quan tới latency, freshness, auditability, cost và recoverability như thế nào.

Một outcome quan trọng cho DS là chuyển từ tư duy “model có metric tốt” sang “hệ thống dùng model có vận hành tốt”. Đây là bước chuyển rất lớn: bạn không bỏ modelling, nhưng bạn đặt modelling vào một hệ production có boundary, contract, monitoring và rollback.

Vì sao cần nói rõ mục tiêu?

Một sai lầm phổ biến khi học SA là cố học “tất cả”: SOLID, DDD, CQRS, Event Sourcing, Saga pattern, Outbox pattern, hexagonal, clean architecture, onion architecture, BFF, sidecar, service mesh, mesh app... Mỗi chủ đề lại dẫn tới 5 chủ đề khác. Không ai có thể master tất cả, và cố làm thế chỉ dẫn tới sự nông cạn ở mọi chỗ.

Khoá này có scope rõ ràng và *cố tình* không dạy mọi thứ. Đọc kỹ phần “Không nằm trong scope” để biết bạn cần học gì sau khoá.

Mục tiêu khoá (Aims)

Sau khi hoàn thành khoá, bạn sẽ:

Mục tiêu 1: Nhận biết và lập luận về architectural decisions

Bạn sẽ phân biệt được:

- Architectural decision vs Design decision vs Implementation detail (Bài 1.2 và Phần 3 build kỹ).
- Architectural characteristic (cái bạn tối ưu) vs Architectural style (cách bạn tổ chức để tối ưu).
- Trade-off đa chiều: vì sao “best architecture” không tồn tại trong vacuum.

Mục tiêu 2: Áp dụng SOLID và các nguyên lý thiết kế cấp module

Phần 2 dạy bạn 5 nguyên lý SOLID kèm cohesion-coupling. Sau phần này, bạn:

- Nhận ra anti-pattern vi phạm SOLID trong code thực.
- Refactor được code vi phạm SRP/OCP/LSP/ISP/DIP về dạng tuân thủ.
- Hiểu được khi nào áp dụng SOLID là *over-engineering* (vì SOLID không miễn phí).

Mục tiêu 3: Biết 9 architecture styles phổ biến

Phần 5-6 đi qua 9 style chính: Layered, Pipeline, Microkernel, Monolithic (đối chiếu), Service-based, Microservices, Event-Driven, Space-Based, và một số biến thể. Với mỗi style bạn sẽ biết:

- Topology cốt lõi (vẽ được trên giấy).
- Quality attributes mà nó tối ưu (vd: Microservices tối ưu scalability/deployability, sacrificing simplicity).

- Khi nào dùng, khi nào không.
- Cách evolve giữa các style (vd: monolith ↔ service-based ↔ microservices).

Mục tiêu 4: Đọc và viết documentation kiến trúc chuẩn

Phần 7 dạy 3 loại view (Module, Component-and-Connector, Allocation), đây là chuẩn de-facto của ngành. Sau phần này bạn:

- Đọc được architecture document bất kỳ và nhận ra đó là loại view nào.
- Viết được architecture document cho hệ mình thiết kế, dùng đúng notation chuẩn.
- Biết khi nào dùng ADR (Architecture Decision Record) và viết được một ADR đúng cách.

Mục tiêu 5: Áp dụng vào case study thực

Phần 8 đi qua 3 case study chi tiết (UAMS, Smart City Traffic Detection, Production ML Feature Store) và một bộ bài tập tổng hợp. Sau phần này bạn:

- Tự thiết kế kiến trúc end-to-end cho hệ cỡ trung (50-500k user) dựa trên requirement.
- Trình bày được kiến trúc đó cho người không-kỹ-thuật hiểu (vd: PM, business stakeholder).
- Defend được lựa chọn trước câu hỏi “vì sao không dùng X?”.

Mục tiêu 6: Áp dụng nền tảng vào advanced domains

Phần 9 đưa bạn qua bốn seminar nâng cao: IoT, Web3, MLOps và Digital Twin. Sau phần này bạn:

- Nhìn được constraint đặc thù của từng domain và dịch chúng thành Quality Attributes.
- Biết vì sao các hệ hiện đại thường là style mix, không phải một style duy nhất.
- Đọc và phản biện được architecture proposal cho IoT platform, Web3 app, MLOps platform hoặc Digital Twin system.

Kết quả học tập (Learning Outcomes)

Kết quả học tập là phiên bản đo được của mục tiêu. Sau khi hoàn thành khoá, bạn có thể:

Mã	Kết quả	Phần dạy
LO1	Định nghĩa SA và phân biệt architectural vs design decisions	1, 3
LO2	Áp dụng 5 nguyên lý SOLID khi review/refactor code	2
LO3	Phân tích trade-off khi chọn architecture cho ứng dụng cho trước	3, 4
LO4	Xác định 5-10 architecture characteristics ưu tiên cho hệ mới	4
LO5	Mô tả topology và trade-off của 9 architecture styles phổ biến	5, 6
LO6	Chọn architecture style phù hợp cho một dự án dựa trên QA priorities	5, 6, 8

Mã	Kết quả	Phần dạy
LO7	Vẽ Module View, C&C View, Allocation View cho hệ thiết kế	7
LO8	Viết Architecture Decision Record (ADR) cho một quyết định cụ thể	7
LO9	Thiết kế end-to-end kiến trúc cho hệ cỡ trung trong 2-3 giờ	8
LO10	Đọc, đánh giá, và phản biện architecture của hệ có sẵn	All
LO11	Phân tích architecture cho advanced domains như IoT, Web3, MLOps, Digital Twin	9

Bộ LO này khá tham vọng. Đạt được toàn bộ đòi hỏi 30-50 giờ học nghiêm túc + 10-20 giờ thực hành. Đừng vội nản nếu sau lần đọc đầu tiên chưa đạt, kiến trúc là kỹ năng tích lũy.

Không nằm trong scope

Khoá này cố tình bỏ ra ngoài các chủ đề sau (để giữ scope khả thi):

1. Domain-Driven Design (DDD)

DDD là một methodology để mô hình hoá domain phức tạp. Nó liên quan tới SA nhưng là một body of knowledge riêng (sách của Eric Evans ~500 trang). Học sau khoá này: đọc *Domain-Driven Design Distilled* (Vaughn Vernon) cho phiên bản gọn, hoặc *Implementing Domain-Driven Design* (Vaughn Vernon) cho phiên bản đầy đủ.

2. CQRS, Event Sourcing, Saga, Outbox pattern

Đây là các pattern phổ biến trong distributed systems hiện đại nhưng thuộc về *pattern level*, không phải *style level*. Khoá này dừng ở style. Học sau: *Microservices Patterns* (Chris Richardson) là tài liệu tốt nhất cho các pattern này.

3. Hexagonal / Clean / Onion Architecture

Ba “kiến trúc” này thực ra là cùng một idea (separation of business logic khỏi infrastructure) đóng gói khác nhau. Quan trọng nhưng overlap nhiều với SOLID + layered architecture. Học sau: chương 22-24 của *Clean Architecture* (Robert C. Martin).

4. Cloud-native specifics

Service mesh (Istio, Linkerd), serverless (Lambda, Cloud Run), container orchestration (Kubernetes), API gateway... đều là các topic con của cloud architecture. Khoá này chỉ nhắc đến khi cần trong Phần 6. Học sau: *Cloud Native Patterns* (Cornelia Davis) hoặc tài liệu chính thống của cloud provider.

5. Performance engineering chi tiết

Khoá có nói về performance là một quality attribute, nhưng không đi sâu vào: caching strategies, database tuning, load testing methodology, profiling tools. Học sau: *Designing Data-Intensive Applications* (Martin Kleppmann), sách gối đầu giường về scaling.

6. Security architecture chi tiết

Tương tự, security được nhắc đến nhưng không sâu. Nếu quan tâm security thực sự, học một khoá riêng hoặc xem qua [Software Security Foundation](#), sister course của khoá này.

Lộ trình học tiếp sau khoá

Sau khi hoàn thành khoá này, đây là lộ trình gợi ý theo focus area:

Nếu bạn đi hướng Architect chuyên nghiệp

1. Đọc *Fundamentals of Software Architecture* (Mark Richards, Neal Ford), sách chính của ngành.
2. Đọc *Software Architecture: The Hard Parts* (Neal Ford, Mark Richards), phần distributed deeply.
3. Học DDD qua *Domain-Driven Design Distilled* (Vaughn Vernon).
4. Bắt đầu viết ADR thật trong dự án của bạn.

Nếu bạn đi hướng Tech Lead

1. Đọc *The Software Architect Elevator* (Gregor Hohpe), về vai trò architect trong tổ chức.
2. Đọc *Team Topologies* (Skelton, Pais), về cách tổ chức team xung quanh architecture.
3. Practice trade-off analysis trên dự án thật của team mình.

Nếu bạn đi hướng Distributed Systems

1. Đọc *Designing Data-Intensive Applications* (Martin Kleppmann), bible của ngành.
2. Học Kafka, gRPC, distributed consensus (Raft, Paxos).
3. Practice với một hệ thật (vd: build một service mesh nhỏ).

Nếu bạn đi hướng Cloud Architect

1. Lấy chứng chỉ AWS Solutions Architect Associate hoặc Google Professional Cloud Architect.
2. Đọc *Cloud Native Patterns* (Cornelia Davis).
3. Practice với Kubernetes thật + ít nhất một managed service mesh.

Tóm tắt

- Khoá có 5 mục tiêu chính, đo được qua 10 learning outcomes.
- Khoá *cố tình* không dạy DDD, CQRS, hexagonal, cloud-native deeply, performance/security deeply.
- Sau khoá, có 4 lộ trình tiếp theo tùy focus area.
- Đạt được toàn bộ LO đòi hỏi 30-50 giờ học + thực hành.

Bài tiếp: [Lộ trình đọc tài liệu](#), chọn cách đọc phù hợp với bạn.

1.4 Lộ trình đọc tài liệu

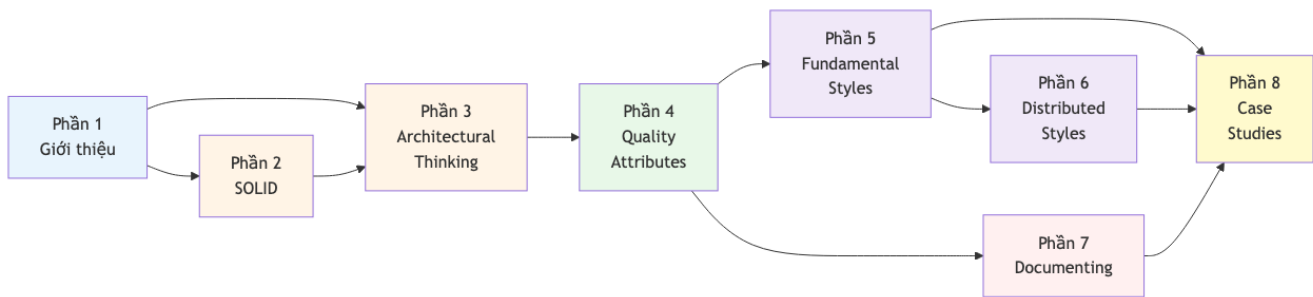
Tóm tắt một dòng: Không phải ai cũng nên đọc khoá theo cùng một thứ tự. Nếu bạn là Data Scientist, hãy ưu tiên Quality Attributes, Pipeline Architecture, Event-Driven Architecture và Documenting trước khi đi sâu vào microservices.

Trước khi chọn lộ trình

Software Architecture là một môn học có tính mạng lưới. Bạn không học một khái niệm rồi đóng lại, mà liên tục quay lại khái niệm đó ở mức sâu hơn. Ví dụ, ban đầu bạn học **cohesion/coupling** ở cấp class. Sang Phần 3, bạn gặp lại nó ở cấp module. Sang Phần 6, bạn gặp lại ở cấp service. Sang Phần 7, bạn phải vẽ được nó thành diagram.

Vì vậy, lộ trình đọc không chỉ là thứ tự file. Nó là cách giảm tải nhận thức. Người đã quen backend có thể đọc thẳng từ Phần 1 tới Phần 8. Người đến từ Data Science nên đi qua một số điểm cầu trước, vì các khái niệm như interface, bounded context, deployment view hoặc distributed transaction không phải vocabulary thường ngày của DS.

Sơ đồ phụ thuộc giữa các phần



Hình 1: Sơ đồ 1

Quy ước màu:

- **Xanh nhạt:** warm-up, đọc nhẹ để dựng khung.
- **Vàng cam:** design level, từ code tốt tới tư duy kiến trúc.
- **Xanh lá:** characteristics, tức cái hệ thống cần tối ưu.
- **Tím:** structural styles, tức cách tổ chức hệ thống.
- **Hồng:** communication, tức cách truyền đạt kiến trúc.
- **Vàng:** application, tức case study và bài tập.

Lộ trình A: Chuyên sâu đầy đủ

Đối tượng: người muốn học nền tảng Software Architecture một cách nghiêm túc, có thể là developer, Tech Lead tương lai, ML Engineer muốn chuyển sang platform, hoặc học viên cao học.

Tuần	Phần	Thời gian	Mục tiêu
1	Phần 1	1-2h	Set up vocabulary và framework tư duy
1-2	Phần 2	6-10h	Từ code khó sửa sang code có boundary rõ

Tuần	Phần	Thời gian	Mục tiêu
3	Phần 3	4-6h	Biết phân biệt architecture decision và design decision
4	Phần 4	4-6h	Biết chọn và đo Quality Attributes
5	Phần 5	5-7h	Nắm monolith, layered, pipeline, microkernel
6	Phần 6	5-7h	Nắm service-based, microservices, event-driven, space-based
7	Phần 7	4-6h	Vẽ 3 view và viết ADR
8	Phần 8	6-10h	Áp toàn bộ vào case study
9	Resources	1-2h	Ôn lại bằng summary, glossary, checklist

Tổng: khoảng 35-55 giờ. Đừng cố đọc hết trong một cuối tuần. Tốt nhất là đọc mỗi phần rồi áp vào một hệ thống bạn biết: một web app, một data pipeline, một model serving system, hoặc một project công ty.

Lộ trình B: Data Scientist to Software Architecture

Đối tượng: Data Scientist, ML Engineer, Data Engineer muốn hiểu architecture để đưa model và data pipeline vào production.

Vì sao path này khác?

Nếu bạn đến từ DS, bạn đã quen với pipeline, data quality, metric, experiment, model artifact. Nhưng bạn có thể chưa quen với cách software team nghĩ về deployment, interface contract, module boundary, availability, rollback, observability. Path này tận dụng cái bạn đã biết trước, rồi nối sang khái niệm mới.

Thứ tự	Bài	Thời gian	Bạn học được gì
1	1.2 Software Architecture là gì?	45m	Vì sao batch vs online inference là decision kiến trúc
2	3.3 Trade-off Analysis	1.5h	Cân accuracy, latency, freshness, cost, explainability
3	4.1 Quality Attributes	1h	Chuyển từ model metric sang system metric
4	4.2 FR vs NFR	1h	Phân biệt “predict được” với “predict ổn trong production”

Thứ tự	Bài	Thời gian	Bạn học được gì
5	5.4 Pipeline Architecture	2h	Map ETL/ML pipeline sang filter-and-pipe architecture
6	6.4 Event-Driven Architecture	2h	Hiểu streaming features, Kafka, async workflows
7	6.2 Service-based Architecture	1.5h	Tổ chức feature service, training service, serving service
8	7.3 C&C Views	1h	Vẽ runtime flow của online inference
9	8.3 Smart City Traffic	2h	Case real-time data/ML system end-to-end
10	Lộ trình cho Data Scientist	30m	Checklist và bài tập riêng cho DS

Tổng: khoảng 13-16 giờ. Sau path này, bạn nên tự tin hơn khi bàn với backend/platform team về feature store, inference latency, model monitoring, hoặc batch vs online architecture.

Lộ trình C: Refresh nhanh

Đối tượng: đã biết software architecture hoặc system design, muốn refresh nhanh để dùng trong công việc.

Thứ tự	Tài liệu	Thời gian
1	Course Summary	1-2h
2	Cross-reference	30m
3	Phần 4 overview + Phần 5/6 overview	1-2h
4	Bài cụ thể đang cần	tuỳ nhu cầu
5	Glossary	30m

Path này không phù hợp nếu bạn mới hoàn toàn. Nó giống dùng bản đồ khi bạn đã biết thành phố, không phải tour guide cho lần đầu.

Lộ trình D: Tra cứu on-demand

Đối tượng: người đã làm dự án thật và chỉ muốn check nhanh một concept.

- Cần thuật ngữ: mở [Glossary](#).
- Cần xem concept liên quan nhau: mở [Cross-reference](#).
- Cần DS-specific path: mở [Lộ trình cho Data Scientist](#).
- Cần review chất lượng nội dung theo góc DS: mở [Review nội dung cho Data Scientist](#).

Shortcut paths theo focus area

Shortcut 1: Tôi muốn hiểu microservices

1. [1.2 Software Architecture là gì?](#), 30 phút.

2. [3.3 Trade-off Analysis](#), 1.5h.
3. [4.1 Quality Attributes](#), 1h.
4. [5.2 Monolithic vs Distributed](#), 1h.
5. [6.2 Service-based](#), 1h.
6. [6.3 Microservices](#), 2h.
7. [6.4 Event-Driven](#), 2h.

Điểm quan trọng: hãy đọc service-based trước microservices. Nếu bỏ qua bước này, bạn rất dễ nghĩ chỉ có hai cực: monolith hoặc microservices. Thực tế nhiều hệ tốt nằm ở giữa.

Shortcut 2: Tôi muốn document architecture cho team

1. [1.2 Software Architecture là gì?](#), 30 phút.
2. [4.3 Identifying characteristics](#), 1h.
3. [7.1 Documenting overview](#), 1h.
4. [7.2 Module Views](#), 1.5h.
5. [7.3 C&C Views](#), 1.5h.
6. [7.4 Allocation Views](#), 1.5h.

Nếu bạn làm ML/data platform, hãy practice bằng một flow cụ thể: online prediction request đi qua API, feature lookup, model inference, logging, monitoring như thế nào.

Shortcut 3: Tôi muốn refactor code DS/ML cho dễ maintain

1. [2.2 Cohesion và Coupling](#), 1.5h.
2. [2.3 SRP](#), 1h.
3. [2.4 OCP](#), 1h.
4. [2.7 DIP](#), 1h.
5. [3.4 Modularity](#), 1.5h.
6. Practice: tách một notebook training thành package có data, features, training, evaluation, serving, 3-5h.

Shortcut 4: Tôi muốn thiết kế production ML system

1. [4.1 Quality Attributes](#)
2. [5.4 Pipeline Architecture](#)
3. [6.4 Event-Driven Architecture](#)
4. [6.2 Service-based Architecture](#)
5. [7.3 C&C Views](#)
6. [8.3 Smart City Traffic](#)
7. [Lộ trình cho Data Scientist](#)

Mục tiêu cuối path này: bạn vẽ được kiến trúc cho churn prediction, fraud detection, recommendation serving hoặc ML monitoring platform.

Lưu ý khi học

Đừng chỉ đọc, hãy vẽ

Software Architecture là kỹ năng applied. Đọc xong một style mà không vẽ topology thì rất dễ tưởng mình hiểu nhưng khi gặp project thật lại bí. Sau mỗi bài ở Phần 5/6, hãy vẽ lại bằng tay:

- Component nào nhận input?
- Component nào lưu state?
- Data đi qua đâu?
- Failure xảy ra ở đâu?
- Nếu load tăng 10 lần, scale chỗ nào?

Với DS, hãy dùng project quen thuộc của bạn. Ví dụ: một pipeline train churn model hằng ngày. Vẽ data source, feature transformation, training job, model registry, batch scoring, CRM export, monitoring.

Đừng bị quyến rũ bởi trends

Microservices, event-driven, feature store, vector database, service mesh đều có chỗ dùng đúng. Nhưng không cái nào là thuốc chữa bách bệnh. Một cron job đơn giản có monitoring tốt đôi khi thắng một Kafka pipeline phức tạp nhưng không ai vận hành nổi.

Quay lại Phần 4 nhiều lần

Nếu bạn thấy phân vân giữa hai architecture style, thường là vì bạn chưa nói rõ Quality Attributes. Muốn freshness cao hay cost thấp? Muốn latency thấp hay explainability cao? Muốn consistency mạnh hay availability cao? Phần 4 là nơi giúp bạn biến tranh luận cảm tính thành quyết định có lý do.

Tiếp theo

Nếu bạn đi theo path chuẩn, bắt đầu với [Phần 1: Tổng quan](#). Nếu bạn đến từ Data Science, mở [Lộ trình cho Data Scientist](#) trước, rồi quay lại các bài trong path B.